

COMPTE RENDU

Administration Réseaux et Programmation Système - TP1 -Construction
d'un Réseau Virtuel Étendu : Test de l'ARP Statique et Dynamique
3e année Cybersécurité - École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha
Filière : Cybersécurité
Année : 2025/2026
Enseignant : M.Bakhouya & M.Aqachtoul
Date : 28 mars 2026

Table des matières

1	Partie 1 : Construction du réseau virtuel	2
1.1	Objectif et contexte	2
1.2	Tableau de correspondance Cisco / Linux	2
1.3	Topologie du réseau virtuel	2
1.4	Étapes de déploiement	2
2	Partie 2 : Tests de connectivité	5
2.1	Scénario 1 — Existence des namespaces	5
2.2	Scénario 2 — Interfaces actives (UP)	5
2.3	Scénario 3 — Configuration IP	6
2.4	Scénario 4 — Connectivité locale LAN1 (via SW1/br-lan1)	6
2.5	Scénario 5 — Connectivité locale LAN2 (via SW2/br-lan2)	6
2.6	Scénario 6 — Isolation inter-LAN (forwarding désactivé)	7
2.7	Scénario 7 — Routage inter-LAN (forwarding réactivé)	7
2.8	Scénario 8 — Lien Passerelle ↔ FAI (WAN)	7
2.9	Scénario 9 — Test bout-en-bout PC → FAI (sans NAT)	7
2.10	Tables de routage	8
3	Partie 3 : Observations ARP & Commutation	8
3.1	Étape 1 — État ARP initial vide	8
3.2	Étape 2 — Création dynamique de l'ARP après ping	8
3.3	Étape 3 — Vieillessement ARP (comportement dynamique)	9
3.4	Étape 4 — Capture tcpdump du trafic ARP	9
3.5	Étape 5 — Entrée ARP statique permanente	11
3.6	Isolation des domaines de broadcast (LAN1 vs LAN2)	12
	Conclusion	12

1 Partie 1 : Construction du réseau virtuel

1.1 Objectif et contexte

Ce TP étend la topologie du TP0 (PC1/PC2 point-à-point) vers un réseau LAN commuté complet. Deux LANs distincts sont simulés à l'aide de **bridges Linux** jouant le rôle de commutateurs Ethernet, avec six PCs, deux switches virtuels (SW1/SW2) et une passerelle commune.

Objectifs principaux :

- Construire un réseau virtuel étendu avec bridges Linux
- Observer et comparer le comportement ARP **dynamique** vs **statique**
- Vérifier l'isolation des domaines de broadcast entre LAN1 et LAN2

1.2 Tableau de correspondance Cisco / Linux

Composant Cisco	Équivalent Linux
PC	Namespace (PC1-PC6)
Commutateur Ethernet	Namespace + bridge Linux (SW1/br-lan1, SW2/br-lan2)
Routeur Passerelle	Namespace GATEWAY (forwarding IP activé)
Routeur FAI	Namespace ISP
Interface G0/x	Interface veth (paire virtuelle)
Port de switch	Interface veth attachée au bridge (master br-lan)
Bloc IP public	Adresse IP sur veth3 (209.165.200.226/27)
Nuage Internet	Namespace ISP

1.3 Topologie du réseau virtuel

Équipement	Interface	Adresse IP	Réseau
PC1	veth-PC1-pc	192.168.10.10	192.168.10.0/24
PC3	veth-PC3-pc	192.168.10.11	192.168.10.0/24
PC5	veth-PC5-pc	192.168.10.12	192.168.10.0/24
PC2	veth-PC2-pc	192.168.20.10	192.168.20.0/24
PC4	veth-PC4-pc	192.168.20.11	192.168.20.0/24
PC6	veth-PC6-pc	192.168.20.12	192.168.20.0/24
GATEWAY	gw-lan1	192.168.10.1	192.168.10.0/24
GATEWAY	gw-lan2	192.168.20.1	192.168.20.0/24
GATEWAY	veth3	209.165.200.226	209.165.200.224/27
ISP	isp1	209.165.200.225	209.165.200.224/27

1.4 Étapes de déploiement

Étape 1 : Création des namespaces

Listing 1 – Création des 10 namespaces

```
1 for ns in PC1 PC2 PC3 PC4 PC5 PC6 SW1 SW2 GATEWAY ISP; do
2     sudo ip netns add "$ns"
3 done
```

Étape 2 : Création des bridges Linux dans SW1 et SW2

Listing 2 – Création des bridges br-lan1 et br-lan2

```
1 # Bridge LAN1 dans SW1
2 sudo ip netns exec SW1 ip link add br-lan1 type bridge
3 sudo ip netns exec SW1 ip link set br-lan1 up
4
5 # Bridge LAN2 dans SW2
6 sudo ip netns exec SW2 ip link add br-lan2 type bridge
7 sudo ip netns exec SW2 ip link set br-lan2 up
```

Étape 3 : Création des paires veth et assignation aux namespaces

Listing 3 – Paires veth pour LAN1, LAN2, GATEWAY et ISP

```
1 # LAN1 : PC1, PC3, PC5 <-> SW1
2 sudo ip link add veth-PC1-pc type veth peer name veth-PC1-sw
3 sudo ip link add veth-PC3-pc type veth peer name veth-PC3-sw
4 sudo ip link add veth-PC5-pc type veth peer name veth-PC5-sw
5
6 # LAN2 : PC2, PC4, PC6 <-> SW2
7 sudo ip link add veth-PC2-pc type veth peer name veth-PC2-sw
8 sudo ip link add veth-PC4-pc type veth peer name veth-PC4-sw
9 sudo ip link add veth-PC6-pc type veth peer name veth-PC6-sw
10
11 # GATEWAY <-> SW1 et SW2
12 sudo ip link add gw-lan1 type veth peer name gw-lan1-sw
13 sudo ip link add gw-lan2 type veth peer name gw-lan2-sw
14
15 # GATEWAY <-> ISP (WAN)
16 sudo ip link add veth3 type veth peer name isp1
17
18 # Assignation aux namespaces
19 sudo ip link set veth-PC1-pc netns PC1 ; sudo ip link set veth-
    PC1-sw netns SW1
20 sudo ip link set veth-PC3-pc netns PC3 ; sudo ip link set veth-
    PC3-sw netns SW1
21 sudo ip link set veth-PC5-pc netns PC5 ; sudo ip link set veth-
    PC5-sw netns SW1
22 sudo ip link set veth-PC2-pc netns PC2 ; sudo ip link set veth-
    PC2-sw netns SW2
23 sudo ip link set veth-PC4-pc netns PC4 ; sudo ip link set veth-
    PC4-sw netns SW2
24 sudo ip link set veth-PC6-pc netns PC6 ; sudo ip link set veth-
    PC6-sw netns SW2
25 sudo ip link set gw-lan1 netns GATEWAY ; sudo ip link set gw-
    lan1-sw netns SW1
26 sudo ip link set gw-lan2 netns GATEWAY ; sudo ip link set gw-
    lan2-sw netns SW2
27 sudo ip link set veth3 netns GATEWAY ; sudo ip link set isp1
    netns ISP
```

Étape 4 : Attachement des interfaces aux bridges

Listing 4 – Rattachement des ports au bridge (mode switch)

```
1 # SW1 (br-lan1) : PC1, PC3, PC5 + cote GATEWAY
2 for iface in veth-PC1-sw veth-PC3-sw veth-PC5-sw gw-lan1-sw; do
3     sudo ip netns exec SW1 ip link set "$iface" up
4     sudo ip netns exec SW1 ip link set "$iface" master br-lan1
5 done
6
7 # SW2 (br-lan2) : PC2, PC4, PC6 + cote GATEWAY
8 for iface in veth-PC2-sw veth-PC4-sw veth-PC6-sw gw-lan2-sw; do
9     sudo ip netns exec SW2 ip link set "$iface" up
10    sudo ip netns exec SW2 ip link set "$iface" master br-lan2
11 done
```

Étape 5 : Adressage IP, routage et forwarding

Listing 5 – Adressage IP, routes par défaut et activation forwarding

```
1 # LAN1
2 sudo ip netns exec PC1 ip addr add 192.168.10.10/24 dev veth-PC1-
   pc
3 sudo ip netns exec PC3 ip addr add 192.168.10.11/24 dev veth-PC3-
   pc
4 sudo ip netns exec PC5 ip addr add 192.168.10.12/24 dev veth-PC5-
   pc
5 for iface in veth-PC1-pc veth-PC3-pc veth-PC5-pc; do
6     ns=$(echo $iface | sed 's/veth-\(PC[0-9]\)\.*/\1/')
7     sudo ip netns exec $ns ip link set $iface up
8     sudo ip netns exec $ns ip link set lo up
9     sudo ip netns exec $ns ip route add default via 192.168.10.1
10 done
11
12 # LAN2 (même logique pour PC2, PC4, PC6 via 192.168.20.1)
13 sudo ip netns exec PC2 ip addr add 192.168.20.10/24 dev veth-PC2-
   pc
14 sudo ip netns exec PC4 ip addr add 192.168.20.11/24 dev veth-PC4-
   pc
15 sudo ip netns exec PC6 ip addr add 192.168.20.12/24 dev veth-PC6-
   pc
16
17 # GATEWAY
18 sudo ip netns exec GATEWAY ip addr add 192.168.10.1/24 dev gw-
   lan1
19 sudo ip netns exec GATEWAY ip addr add 192.168.20.1/24 dev gw-
   lan2
20 sudo ip netns exec GATEWAY ip addr add 209.165.200.226/27 dev
   veth3
```

```

21 sudo ip netns exec GATEWAY ip route add default via
    209.165.200.225
22 sudo ip netns exec GATEWAY sysctl -w net.ipv4.ip_forward=1
23
24 # ISP
25 sudo ip netns exec ISP ip addr add 209.165.200.225/27 dev isp1

```

2 Partie 2 : Tests de connectivité

2.1 Scénario 1 — Existence des namespaces

Listing 6 – Vérification des 10 namespaces

```

1 hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/
  Admin_Reseau/TP1$ ip netns list
2 ISP (id: 9)
3 GATEWAY (id: 8)
4 SW2 (id: 7)
5 SW1 (id: 6)
6 PC6 (id: 5)
7 PC5 (id: 4)
8 PC4 (id: 3)
9 PC3 (id: 2)
10 PC2 (id: 1)
11 PC1 (id: 0)

```

Les dix namespaces sont présents et identifiés par des IDs distincts.

2.2 Scénario 2 — Interfaces actives (UP)

Listing 7 – Interfaces actives dans chaque namespace

```

1 PC1      -> veth-PC1-pc
2 PC2      -> veth-PC2-pc
3 PC3      -> veth-PC3-pc
4 PC4      -> veth-PC4-pc
5 PC5      -> veth-PC5-pc
6 PC6      -> veth-PC6-pc
7 SW1      -> br-lan1   veth-PC1-sw   veth-PC3-sw   veth-PC5-sw   gw-
    lan1-sw
8 SW2      -> br-lan2   veth-PC2-sw   veth-PC4-sw   veth-PC6-sw   gw-
    lan2-sw
9 GATEWAY  -> gw-lan1   gw-lan2     veth3
10 ISP      -> isp1

```

2.3 Scénario 3 — Configuration IP

Listing 8 – Adressage IP vérifié sur tous les namespaces

1	Namespace	Interface	Adresse IP
2	-----	-----	-----
3	PC1	veth-PC1-pc	192.168.10.10/24
4	PC2	veth-PC2-pc	192.168.20.10/24
5	PC3	veth-PC3-pc	192.168.10.11/24
6	PC4	veth-PC4-pc	192.168.20.11/24
7	PC5	veth-PC5-pc	192.168.10.12/24
8	PC6	veth-PC6-pc	192.168.20.12/24
9	GATEWAY	gw-lan1	192.168.10.1/24
10	GATEWAY	gw-lan2	192.168.20.1/24
11	GATEWAY	veth3	209.165.200.226/27
12	ISP	isp1	209.165.200.225/27

2.4 Scénario 4 — Connectivité locale LAN1 (via SW1/br-lan1)

Listing 9 – Tests de ping intra-LAN1

```
1 $ sudo ip netns exec PC1 ping -c 2 192.168.10.11
2 [SUCCES] PC1 -> PC3 (192.168.10.11)
3 $ sudo ip netns exec PC1 ping -c 2 192.168.10.12
4 [SUCCES] PC1 -> PC5 (192.168.10.12)
5 $ sudo ip netns exec PC3 ping -c 2 192.168.10.10
6 [SUCCES] PC3 -> PC1 (192.168.10.10)
7 $ sudo ip netns exec PC5 ping -c 2 192.168.10.10
8 [SUCCES] PC5 -> PC1 (192.168.10.10)
9 $ sudo ip netns exec PC1 ping -c 2 192.168.10.1
10 [SUCCES] PC1 -> GATEWAY LAN1 (192.168.10.1)
```

Tous les pings aboutissent avec 0% de perte, confirmant que le bridge `br-lan1` commute correctement les trames entre PC1, PC3 et PC5.

2.5 Scénario 5 — Connectivité locale LAN2 (via SW2/br-lan2)

Listing 10 – Tests de ping intra-LAN2

```
1 $ sudo ip netns exec PC2 ping -c 2 192.168.20.11
2 [SUCCES] PC2 -> PC4 (192.168.20.11)
3 $ sudo ip netns exec PC2 ping -c 2 192.168.20.12
4 [SUCCES] PC2 -> PC6 (192.168.20.12)
5 $ sudo ip netns exec PC4 ping -c 2 192.168.20.10
6 [SUCCES] PC4 -> PC2 (192.168.20.10)
7 $ sudo ip netns exec PC6 ping -c 2 192.168.20.10
8 [SUCCES] PC6 -> PC2 (192.168.20.10)
9 $ sudo ip netns exec PC2 ping -c 2 192.168.20.1
10 [SUCCES] PC2 -> GATEWAY LAN2 (192.168.20.1)
```

2.6 Scénario 6 — Isolation inter-LAN (forwarding désactivé)

Listing 11 – Désactivation du forwarding et test d'isolation

```
1 $ sudo ip netns exec GATEWAY sysctl -w net.ipv4.ip_forward=0
2 net.ipv4.ip_forward = 0
3
4 $ sudo ip netns exec PC1 ping -c 2 192.168.20.10
5 [ATTENDU] PC1 -> PC2 : echec correct -- forwarding desactive
```

Ce résultat est **attendu** : sans forwarding IP, GATEWAY ne route plus entre ses interfaces. LAN1 et LAN2 deviennent des domaines de broadcast complètement isolés, ce qui permet d'observer le comportement ARP pur de la couche 2.

2.7 Scénario 7 — Routage inter-LAN (forwarding réactivé)

Listing 12 – Routage inter-LAN après réactivation du forwarding

```
1 $ sudo ip netns exec GATEWAY sysctl -w net.ipv4.ip_forward=1
2
3 $ sudo ip netns exec PC1 ping -c 2 192.168.20.10
4 [SUCCES] PC1 (LAN1) -> PC2 (LAN2) via GATEWAY
5 $ sudo ip netns exec PC1 ping -c 2 192.168.20.11
6 [SUCCES] PC1 (LAN1) -> PC4 (LAN2) via GATEWAY
7 $ sudo ip netns exec PC2 ping -c 2 192.168.10.10
8 [SUCCES] PC2 (LAN2) -> PC1 (LAN1) via GATEWAY
9 $ sudo ip netns exec PC5 ping -c 2 192.168.20.12
10 [SUCCES] PC5 (LAN1) -> PC6 (LAN2) via GATEWAY
```

2.8 Scénario 8 — Lien Passerelle ↔ FAI (WAN)

Listing 13 – Connectivité WAN bidirectionnelle

```
1 $ sudo ip netns exec GATEWAY ping -c 2 209.165.200.225
2 [SUCCES] GATEWAY -> ISP (209.165.200.225)
3 $ sudo ip netns exec ISP ping -c 2 209.165.200.226
4 [SUCCES] ISP -> GATEWAY WAN (209.165.200.226)
```

2.9 Scénario 9 — Test bout-en-bout PC → FAI (sans NAT)

Listing 14 – Echec attendu : PC vers ISP sans NAT

```
1 $ sudo ip netns exec PC1 ping -c 2 209.165.200.225
2 [ATTENDU] PC1 -> ISP : echec correct -- ISP sans route retour
3
4 $ sudo ip netns exec PC2 ping -c 2 209.165.200.225
5 [ATTENDU] PC2 -> ISP : echec correct -- ISP sans route retour
```

ISP ne connaît pas 192.168.x.0/24 : l'Echo Reply est abandonné. Le NAT (TP3) résoudra ce problème en traduisant l'IP source privée.

2.10 Tables de routage

Listing 15 – Tables de routage de tous les namespaces

```
1 PC1 :
2   default via 192.168.10.1 dev veth-PC1-pc
3   192.168.10.0/24 dev veth-PC1-pc proto kernel src 192.168.10.10
4
5 PC2 :
6   default via 192.168.20.1 dev veth-PC2-pc
7   192.168.20.0/24 dev veth-PC2-pc proto kernel src 192.168.20.10
8
9 GATEWAY :
10  default via 209.165.200.225 dev veth3
11  192.168.10.0/24 dev gw-lan1 proto kernel src 192.168.10.1
12  192.168.20.0/24 dev gw-lan2 proto kernel src 192.168.20.1
13  209.165.200.224/27 dev veth3 proto kernel src 209.165.200.226
14
15 ISP :
16  209.165.200.224/27 dev isp1 proto kernel src 209.165.200.225
```

3 Partie 3 : Observations ARP & Commutation

3.1 Étape 1 — État ARP initial vide

Avant toute génération de trafic, les tables ARP de tous les PCs sont vidées avec `ip neigh flush all`.

Listing 16 – Vidage et vérification des tables ARP

```
1 $ sudo ip netns exec PC1 ip neigh flush all
2 $ sudo ip netns exec PC1 ip neigh
3 (vide)
4
5 $ sudo ip netns exec PC3 ip neigh
6 (vide)
7
8 $ sudo ip netns exec PC5 ip neigh
9 (vide)
```

Aucune entrée n'est présente : l'état de départ est propre.

3.2 Étape 2 — Création dynamique de l'ARP après ping

Le forwarding IP est désactivé sur GATEWAY afin d'observer uniquement le comportement de la couche 2 et de l'ARP, sans aucun routage inter-LAN.

Listing 17 – Tables ARP avant et après le premier ping PC1 -j PC3

```
1 --- AVANT ping ---
2 PC1 : (vide)
3 PC3 : (vide)
```

```

4 PC5 : (vide)
5
6 --- APRES ping -c 2 192.168.10.11 ---
7 PC1 :
8   192.168.10.11 dev veth-PC1-pc lladdr ce:4d:a4:ac:c4:fd
      REACHABLE
9 PC3 :
10  192.168.10.10 dev veth-PC3-pc lladdr 52:3e:e2:a4:a1:6e
      REACHABLE

```

Analyse :

1. PC1 ne connaît pas la MAC de 192.168.10.11 $\Rightarrow$ il envoie un **ARP Request broadcast** sur LAN1.
2. PC3 reçoit la requête, reconnaît son IP, et répond par un **ARP Reply unicast** avec sa MAC (ce:4d:a4:ac:c4:fd).
3. Simultanément, PC3 enregistre la MAC de PC1 dans sa propre table ARP : **apprentissage bidirectionnel**.
4. Les deux entrées apparaissent en état REACHABLE (MAC fraîchement confirmée).

3.3 Étape 3 — Vieillessement ARP (comportement dynamique)

Les entrées ARP dynamiques ne sont pas permanentes. En l'absence de trafic, elles évoluent selon les états suivants :

Listing 18 – Cycle de vie d'une entrée ARP dynamique

```

1 Etat immediatement apres ping :
2   192.168.10.11 dev veth-PC1-pc lladdr ce:4d:a4:ac:c4:fd
      REACHABLE
3
4 Apres ~30 secondes sans trafic :
5   192.168.10.11 dev veth-PC1-pc lladdr ce:4d:a4:ac:c4:fd STALE
6
7 Apres ~60 secondes (ou si le gc expire) :
8   (vide -- entree supprimee automatiquement)

```

Signification des états :

- REACHABLE : MAC confirmée récemment, utilisable directement.
- STALE : entrée ancienne ; au prochain paquet, le noyau enverra un ARP de vérification.
- DELAY / PROBE : vérification en cours (NUD — Neighbor Unreachability Detection).
- (vide) : entrée expirée et supprimée par le garbage collector ARP.

Remarque : Une entrée PERMANENT (statique) ne passe jamais par ces états — elle reste indéfiniment jusqu'à suppression manuelle.

3.4 Étape 4 — Capture tcpdump du trafic ARP

Des captures `tcpdump` sont lancées en parallèle sur PC1, PC3, PC5 et le bridge `br-lan1` avant la génération du trafic.

Listing 19 – Lancement des captures tcpdump (en arrière-plan)

```

1 sudo ip netns exec PC1 tcpdump -nni veth-PC1-pc arp -w pc1_arp.
  pcap &
2 sudo ip netns exec PC3 tcpdump -nni veth-PC3-pc arp -w pc3_arp.
  pcap &
3 sudo ip netns exec PC5 tcpdump -nni veth-PC5-pc arp -w pc5_arp.
  pcap &
4 sudo ip netns exec SW1 tcpdump -nni br-lan1 arp -w sw1_br_arp.
  pcap &

```

Listing 20 – Capture ARP sur PC1 – trafic dynamique (pc1_arp_text.txt)

```

1 22:11:33.224594 ARP, Request who-has 192.168.10.11 tell
  192.168.10.10, length 28
2 22:11:33.224701 ARP, Reply 192.168.10.11 is-at ce:4d:a4:ac:c4:fd,
  length 28
3 22:11:35.292870 ARP, Request who-has 192.168.10.12 tell
  192.168.10.10, length 28
4 22:11:35.292944 ARP, Reply 192.168.10.12 is-at a2:20:8a:9b:73:eb,
  length 28

```

Analyse de la capture :

- Ligne 1 : PC1 (192.168.10.10) diffuse un ARP Request broadcast pour localiser PC3 (192.168.10.11).
- Ligne 2 : PC3 répond en unicast avec sa MAC ce:4d:a4:ac:c4:fd.
- Ligne 3 : PC1 envoie ensuite un ARP Request pour PC5 (192.168.10.12).
- Ligne 4 : PC5 répond avec sa MAC a2:20:8a:9b:73:eb.

Tables ARP après les pings (tcpdump terminé) :

Listing 21 – Tables ARP après captures

```

1 PC1 :
2   192.168.10.11 dev veth-PC1-pc lladdr ce:4d:a4:ac:c4:fd
  REACHABLE
3   192.168.10.12 dev veth-PC1-pc lladdr a2:20:8a:9b:73:eb
  REACHABLE
4 PC3 :
5   192.168.10.10 dev veth-PC3-pc lladdr 52:3e:e2:a4:a1:6e DELAY
6 PC5 :
7   192.168.10.10 dev veth-PC5-pc lladdr 52:3e:e2:a4:a1:6e DELAY

```

Table de forwarding MAC du bridge SW1 :

Listing 22 – Table MAC de SW1 – apprentissage par le bridge (bridge fdb show)

```

1 52:3e:e2:a4:a1:6e dev veth-PC1-sw master br-lan1
2 ce:4d:a4:ac:c4:fd dev veth-PC3-sw master br-lan1
3 a2:20:8a:9b:73:eb dev veth-PC5-sw master br-lan1
4 1e:cc:99:1c:5a:cb dev gw-lan1-sw master br-lan1

```

Le bridge a appris l'association MAC → port pour chaque équipement de LAN1. Il peut désormais **commuter en unicast** plutôt que de diffuser en broadcast.

3.5 Étape 5 — Entrée ARP statique permanente

Une entrée ARP statique est ajoutée manuellement sur PC1 pour PC3, en utilisant directement sa MAC adresse.

Listing 23 – Récupération de la MAC de PC3 et ajout de l'entrée statique

```
1 # Recupérer la MAC de PC3
2 $ sudo ip netns exec PC3 ip link show veth-PC3-pc
3   link/ether ce:4d:a4:ac:c4:fd
4
5 # Supprimer l'entree dynamique si elle existe
6 $ sudo ip netns exec PC1 ip neigh del 192.168.10.11 dev veth-PC1-
   pc
7
8 # Ajouter l'entree ARP statique PERMANENT
9 $ sudo ip netns exec PC1 ip neigh add 192.168.10.11 \
10    lladdr ce:4d:a4:ac:c4:fd dev veth-PC1-pc nud permanent
11
12 # Verification
13 $ sudo ip netns exec PC1 ip neigh show
14   192.168.10.11 dev veth-PC1-pc lladdr ce:4d:a4:ac:c4:fd
      PERMANENT
15   192.168.10.12 dev veth-PC1-pc lladdr a2:20:8a:9b:73:eb
      REACHABLE
```

Preuve de l'absence de broadcast ARP depuis PC1 vers PC3 :

Après ajout de l'entrée statique, un tcpdump est lancé sur br-lan1 pendant un ping PC1 → PC3. Le résultat (fichier sw1.static.arp.txt) montre :

Listing 24 – Capture ARP sur SW1 pendant ping PC1 -> PC3 avec ARP statique

```
1 22:11:38.983661 ARP, Request who-has 192.168.10.10 tell
   192.168.10.11, length 28
2 22:11:38.983669 ARP, Reply 192.168.10.10 is-at 52:3e:e2:a4:a1:6
   e, length 28
3 22:11:40.455966 ARP, Request who-has 192.168.10.12 tell
   192.168.10.10, length 28
4 22:11:40.455983 ARP, Request who-has 192.168.10.10 tell
   192.168.10.12, length 28
5 22:11:40.456056 ARP, Reply 192.168.10.12 is-at a2:20:8a:9b:73:
   eb, length 28
6 22:11:40.456060 ARP, Reply 192.168.10.10 is-at 52:3e:e2:a4:a1:6
   e, length 28
```

Observation clé : aucun ARP Request who-has 192.168.10.11 tell 192.168.10.10 n'apparaît dans la capture. PC1 n'a **pas émis de broadcast ARP** pour joindre PC3, car il dispose déjà de son adresse MAC dans son cache permanent.

Les seules requêtes ARP visibles proviennent de PC3 et PC5 qui interrogent PC1 (comportement dynamique normal de leur côté).

Critère	ARP Dynamique	ARP Statique
Création	Automatique via broadcast	Manuelle (<code>ip neigh add</code>)
Expiration	Oui (REACHABLE → STALE → vide)	Non (PERMANENT)
Broadcast ARP	Oui, à chaque expiration	Jamais
Usage	Normal, hôtes connus	Sécurité, performances
Risque	Spoofing ARP possible	Résistant au spoofing

3.6 Isolation des domaines de broadcast (LAN1 vs LAN2)

Après un ping PC1 → PC5 (broadcast ARP sur LAN1), les tables ARP de PC2, PC4 et PC6 sont vérifiées :

Listing 25 – Vérification de l’isolation des domaines de broadcast

```

1 $ sudo ip netns exec PC2 ip neigh
2 (vide -- broadcast LAN1 n''a pas traverse)
3
4 $ sudo ip netns exec PC4 ip neigh
5 (vide -- broadcast LAN1 n''a pas traverse)
6
7 $ sudo ip netns exec PC6 ip neigh
8 (vide -- broadcast LAN1 n''a pas traverse)

```

Conclusion : les bridges Linux `br-lan1` et `br-lan2` constituent deux **domaines de broadcast distincts**. Les ARP Requests de LAN1 restent confinés à SW1 et ne traversent pas vers LAN2 (SW2), exactement comme des VLANs sur des switches physiques.

Conclusion

Ce TP1 a permis d’étendre la topologie du TP0 vers un réseau LAN commuté complet, en exploitant les **bridges Linux** comme commutateurs Ethernet virtuels. Six PCs répartis en deux LANs (192.168.10.0/24 et 192.168.20.0/24) communiquent via les bridges `br-lan1` et `br-lan2`, reliés à une passerelle commune.

Les tests de connectivité ont confirmé le fonctionnement correct à chaque niveau : la commutation de couche 2 au sein de chaque LAN (Scénarios 4 et 5), l’isolation par domaines de broadcast garantie par les bridges (Scénario 6), et le routage inter-LAN via GATEWAY (Scénario 7).

Les observations ARP constituent le cœur de ce TP. La capture `tcpdump` a permis de visualiser concrètement le mécanisme d’**ARP dynamique** : broadcast de découverte, réponse unicast, apprentissage bidirectionnel, et vieillissement automatique des entrées (REACHABLE → STALE → expiration). En parallèle, l’**ARP statique permanent** démontre son avantage : aucun broadcast n’est émis, la MAC est connue d’avance, et l’entrée ne peut pas être corrompue par du spoofing ARP.

Enfin, la table de forwarding MAC du bridge (`bridge fdb show`) illustre l’apprentissage automatique des adresses par le commutateur virtuel, élément fondamental de la commutation Ethernet efficace.